



DATA2DASH

AI-Powered Multi-Agent Research Platform

Course: Mobile Computing

Course Supervisor: Dr. Ahmed Hussein

Team

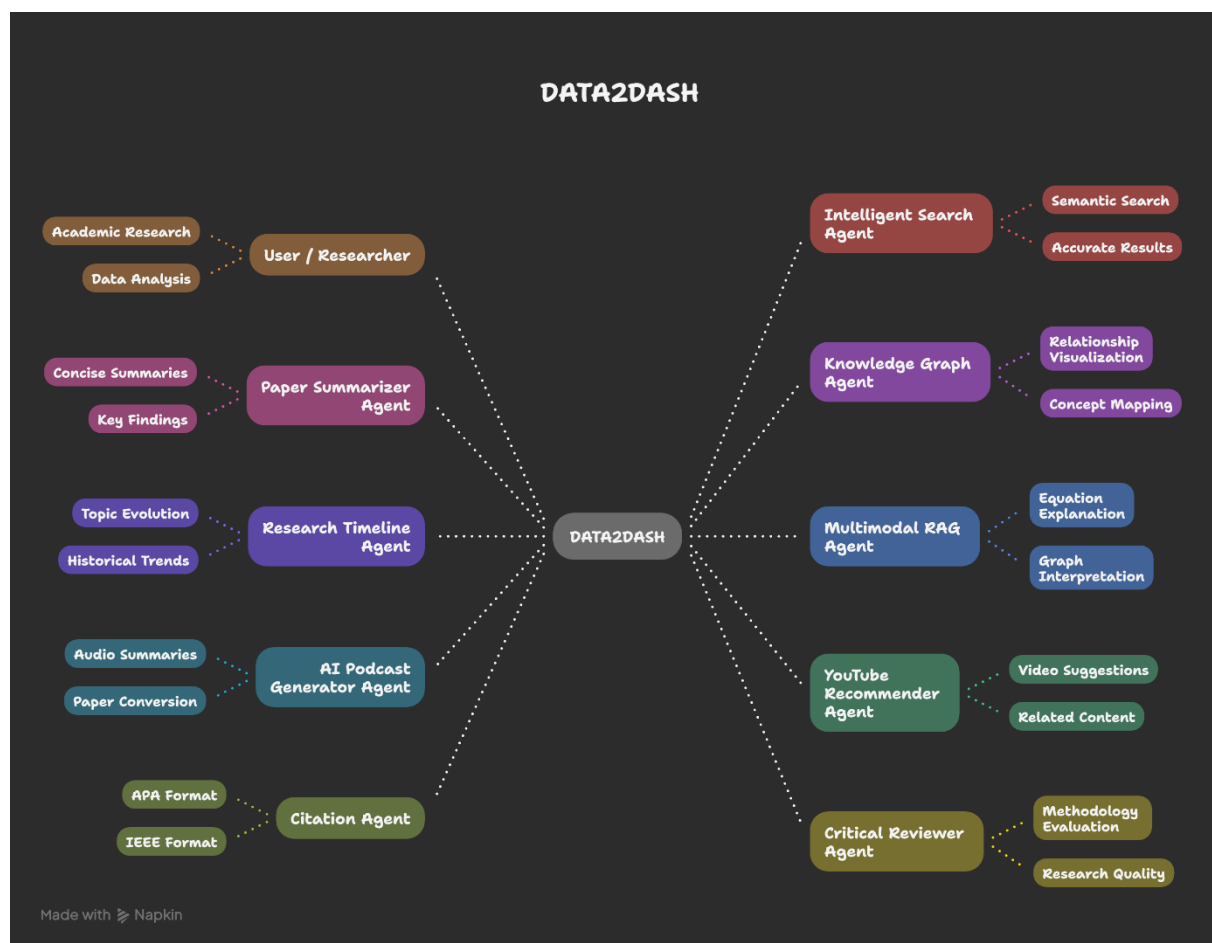
- Abdallah Mohamed Ahmed Ali
- Youssef Ashraf Fathy
- Menna Allah Essam
- Mariam Sherif
- Marwa Hussein Ahmed

Introduction

DATA2DASH is an AI-powered multi-agent research platform designed to help students, researchers, and professionals search, understand, summarize, analyze, and organize academic papers in one unified workspace.

Researchers usually depend on multiple disconnected tools such as Google Scholar for discovery, ChatGPT for explanation, PDF readers for reading, and Word documents for writing notes. This tool fragmentation increases cognitive load and slows down the literature review process. DATA2DASH solves this issue by combining intelligent semantic search, paper summarization, multimodal RAG, knowledge graph visualization, research timeline, podcast generation, YouTube recommendations, citation generation, and critical review in one platform.

The main goal of DATA2DASH is to transform academic research from a manual and time-consuming process into a faster, smarter, and more organized workflow. Instead of only searching and reading papers, users can move toward understanding, analyzing, and discovering insights more efficiently.

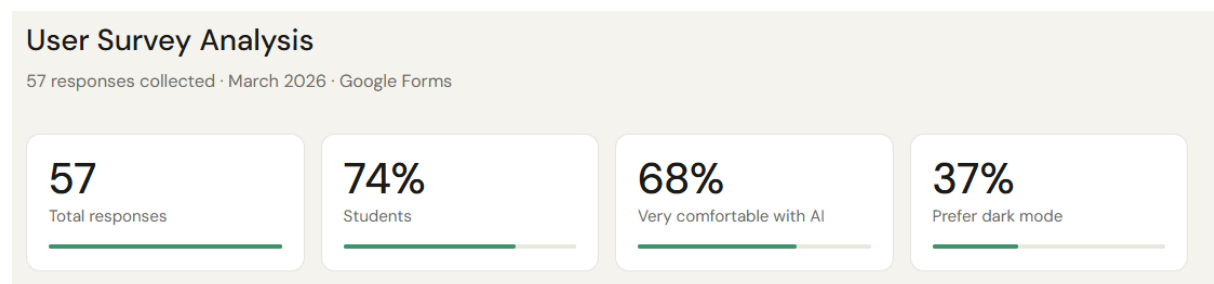


Section 01 — UX Phase

1.1 UX Stage I — Research

The research phase aimed to understand the difficulties users face during academic research and literature review. A user survey was conducted to collect insights about users' current research habits, tools, pain points, preferred features, and interface preferences.

The survey collected **57 responses**. The majority of respondents were students, representing **74%** of the sample. This indicates that the primary target users are students and early researchers who need support in searching, understanding, and organizing academic papers. The survey also showed that many users are comfortable using AI tools, which supports the idea of building an AI-powered research assistant.



1.2 Survey Questions

The survey included questions covering the following areas:

1. What is your current role?
2. How often do you work on literature review or academic research?
3. What tools do you currently use to search for papers?
4. What are the biggest challenges you face while reading or analyzing papers?
5. How comfortable are you with using AI tools?
6. Which features would be most useful in an academic research platform?
7. How much time do you usually spend on literature review per week?
8. What interface theme do you prefer: light mode, dark mode, system default, or toggle option?
9. Do you prefer reading, watching videos, or listening to audio summaries while learning?
10. What would make the research process easier for you?

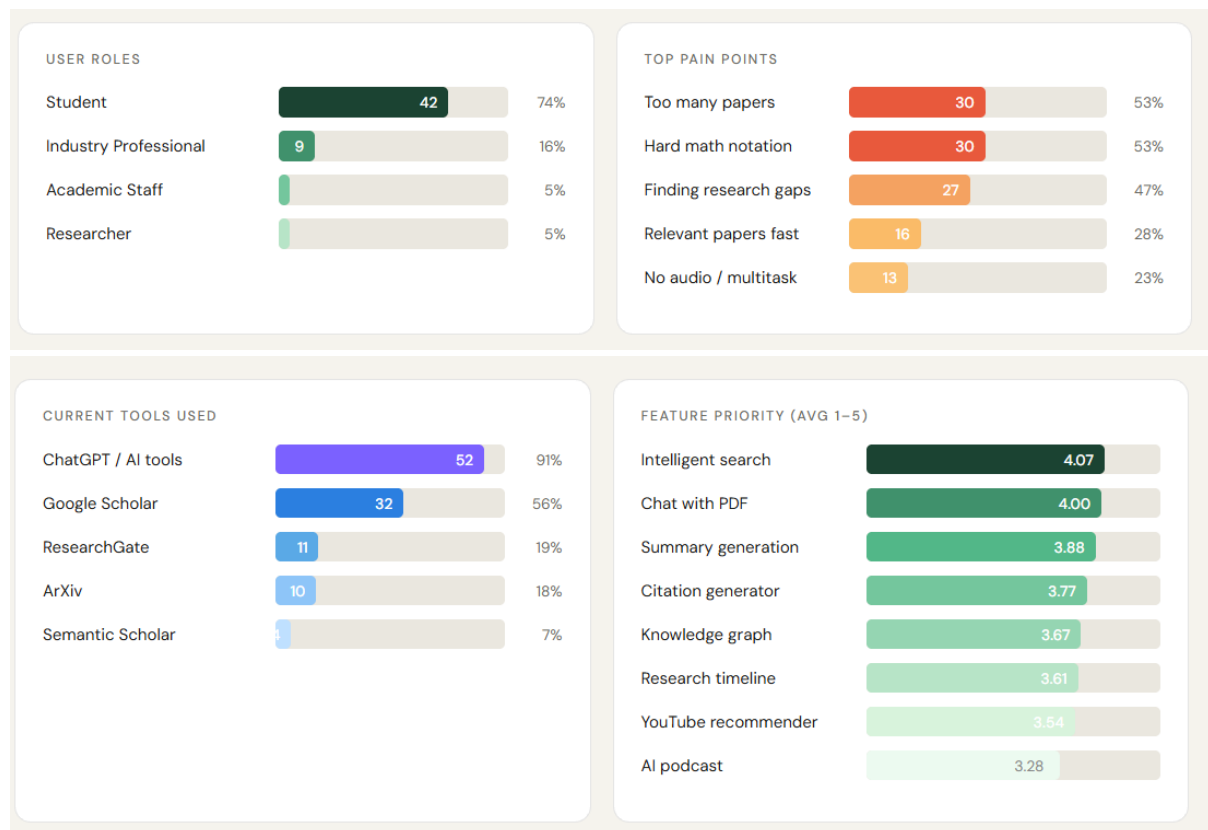
1.3 Survey Analysis

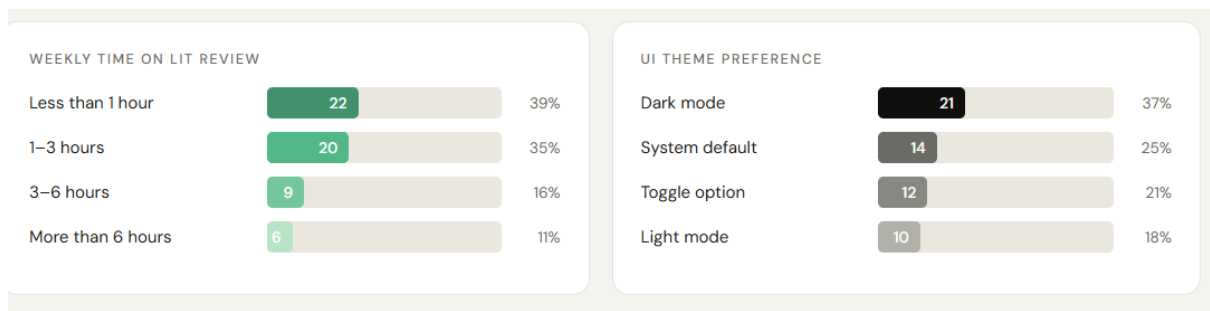
The survey results revealed that most users face several repeated problems during academic research. The most common pain points were information overload, difficulty understanding mathematical notation, difficulty finding research gaps, and the need to find relevant papers quickly.

The results showed that **53%** of respondents struggle with too many papers, while another **53%** struggle with hard mathematical notation. Around **47%** of users reported difficulty finding research gaps, and **28%** needed a faster way to find relevant papers. In addition, **23%** of respondents mentioned the need for audio or multitasking support.

The survey also showed that users currently depend on multiple tools. **91%** use ChatGPT or AI tools, **56%** use Google Scholar, **19%** use ResearchGate, **18%** use ArXiv, and **7%** use Semantic Scholar. This confirms that users already use digital and AI tools, but their workflow is still fragmented across many platforms.

Feature priority results showed that users valued intelligent search the most, with an average rating of **4.07/5**, followed by chat with PDF at **4.00/5**, summary generation at **3.88/5**, citation generator at **3.77/5**, knowledge graph at **3.67/5**, research timeline at **3.61/5**, YouTube recommender at **3.54/5**, and AI podcast at **3.28/5**.





1.4 Research Conclusions

Based on the survey results, the following conclusions were identified:

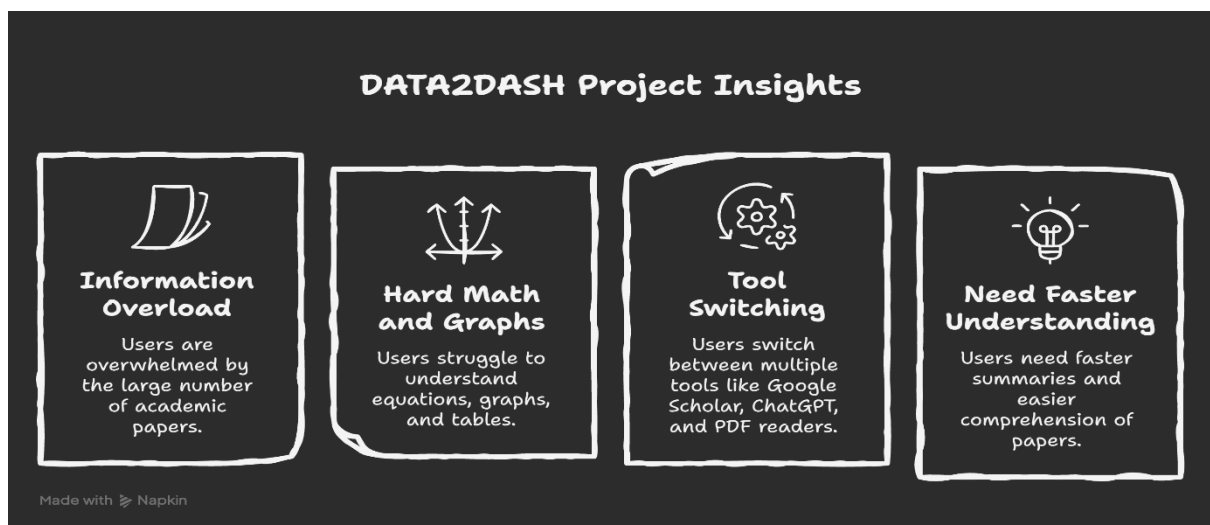
First, students represent the main target audience for DATA2DASH, as they formed the largest percentage of respondents. This means the platform should be simple, guided, and supportive for users who may not have advanced research experience.

Second, users suffer from information overload because they need to review many academic papers in a short time. Therefore, intelligent search and summarization are essential features.

Third, mathematical notation, graphs, and tables make papers harder to understand. This supports the need for a multimodal RAG feature that can explain equations, figures, and tables in a simpler way.

Fourth, users already rely heavily on ChatGPT and other AI tools, but they still use Google Scholar, PDF readers, and other platforms separately. This confirms the need for an all-in-one platform that reduces tool-switching.

Finally, users showed interest in features such as citation generation, knowledge graphs, research timelines, audio learning, and YouTube recommendations. These features can improve the research experience and make learning more flexible.



Pain points and Gains

Pain Points	Expected Gains
Information overload due to too many papers	Find relevant papers faster using intelligent search
Complex mathematics, graphs, and tables	Understand difficult content through multimodal RAG
Hard to identify research gaps	Discover relationships and gaps using knowledge graph and research timeline
Tool fragmentation and switching between platforms	Use one integrated platform for the full research workflow
No audio support for learning on the go	Listen to AI podcast summaries anytime
No quick summary for papers	Get concise summaries before reading the full paper
Manual citation formatting	Generate citations automatically in APA/IEEE
No visual/video support for difficult topics	Get YouTube recommendations for visual explanation

1.5 Empathy Map

The empathy map summarizes what the primary users think, feel, say, and do while conducting academic research.

Thinks:

Users think that there are too many papers to follow and that finding the right papers quickly is difficult. They also feel that mathematical notation in academic papers slows down their understanding.

Feels:

Users feel overwhelmed by the large number of publications and frustrated because they have to use many tools at the same time. However, they are also excited about AI tools because they already use them in their daily learning process.

Says:

Users want semantic search instead of simple keyword matching. They also want a tool that can explain papers, summarize content, and support mobile or flexible learning.

Does:

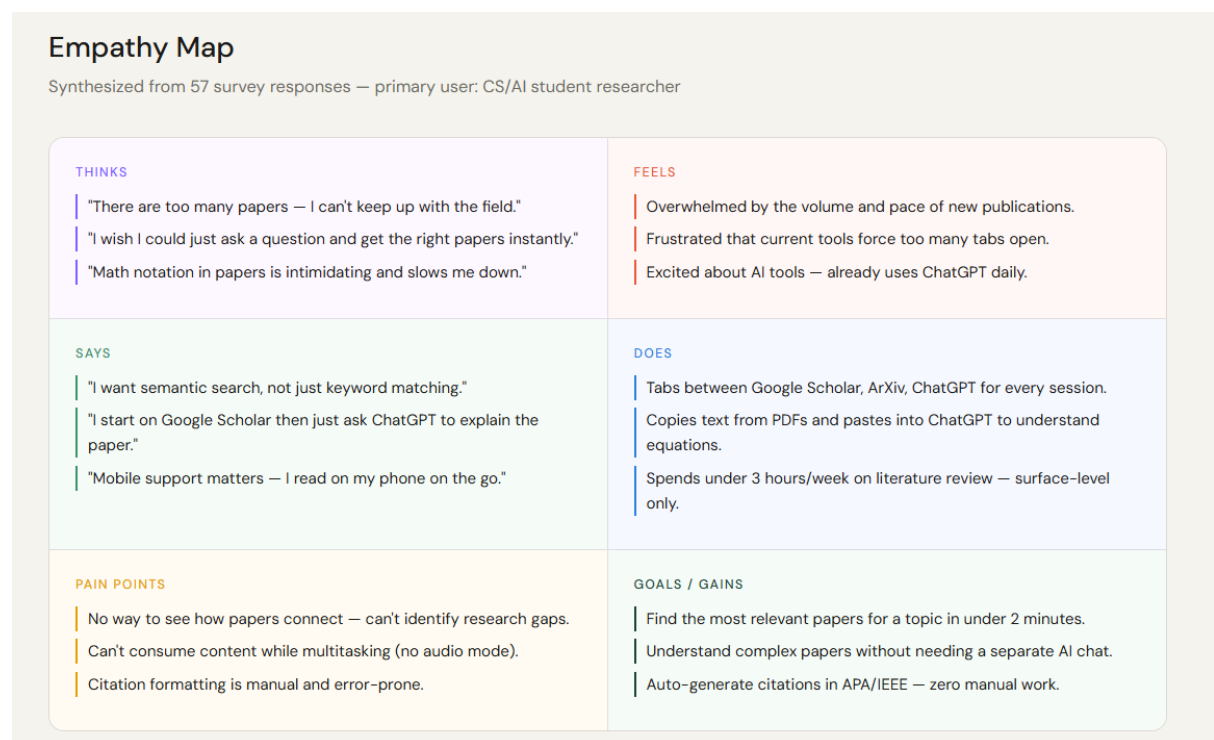
Users switch between Google Scholar, ArXiv, ChatGPT, PDF readers, and writing tools. They often copy text from papers into AI tools to understand difficult sections.

Pain Points:

Users cannot easily see how papers connect, they cannot identify research gaps quickly, they lack audio learning support, and citation formatting is still manual.

Gains:

Users want to find papers quickly, understand complex papers easily, and generate citations automatically.



1.6 Primary Persona

Name: Ahmed Tarek

Age: 22

Role: Computer Engineering Student

Location: Cairo, Egypt

University: Helwan University

Ahmed is a fourth-year computer engineering student interested in AI and cybersecurity. He frequently needs to read academic papers for coursework, research, and

graduation-related topics. He is comfortable using AI tools and often uses ChatGPT to explain difficult concepts.

Ahmed's main problem is that academic research takes too much time. He switches between Google Scholar, ArXiv, ChatGPT, PDF readers, and note-taking tools during one research session. He also struggles with mathematical notation, long papers, and identifying how different papers are connected.

Ahmed needs a platform that helps him search for relevant papers, summarize them, explain difficult equations, generate citations, and visualize relationships between research topics. DATA2DASH supports Ahmed by providing intelligent search, chat with PDF, citation generation, knowledge graph visualization, and AI podcast summaries.

GOALS



1.7 Secondary Persona

Name: Sara Mahmoud

Role: Machine Learning Engineer

User Type: Industry Professional

Sara uses academic research to stay updated with new AI and machine learning methods. She does not have enough time to read full papers deeply, so she needs fast summaries, highly relevant papers, and quick insight extraction. DATA2DASH helps her by providing search, summarization, knowledge graph, and critical review features.

1.8 Wireframes and Sitemap

The following figures show the low-fidelity wireframes created for the Data2Dash AI application. These wireframes were used to plan the main screens, user flow, navigation sidebar, and the layout of the core features before implementing the final UI. The wireframes cover authentication, chat, search, upload, document viewing, and citation-related screens.

Sign In Screen

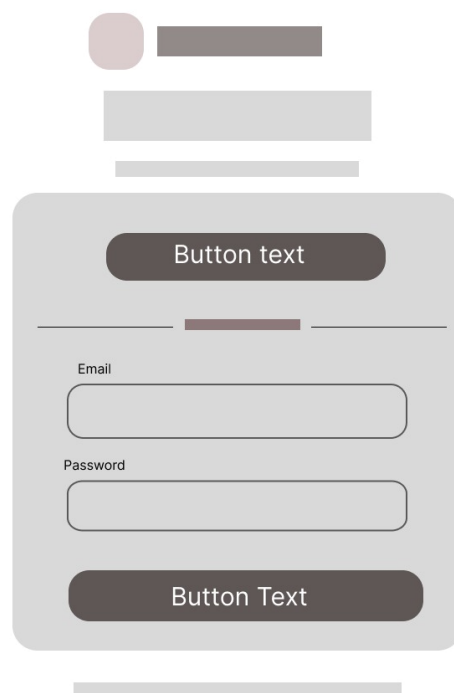


Figure 1.8: Low-fidelity wireframe for the sign-in screen.

This wireframe shows the sign-in screen where existing users can enter their email and password to access the application. It includes a simple authentication form and a main action button to continue to the system.

Create Account Screen

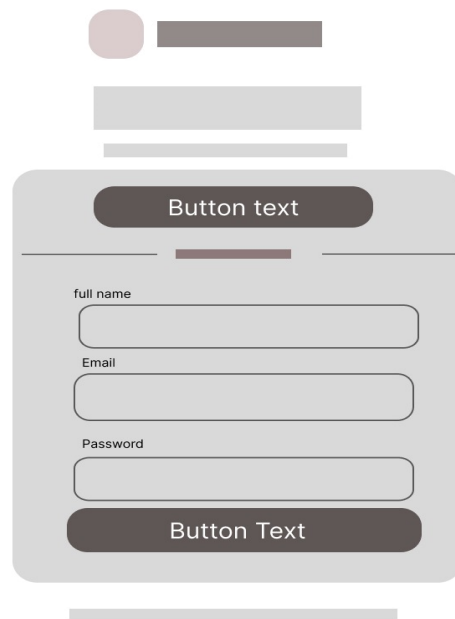


Figure 1.9: Low-fidelity wireframe for the create account screen.

This wireframe shows the registration screen where new users can create an account by entering their full name, email, and password. This screen represents the first step in the user onboarding flow.

Main Chat Screen

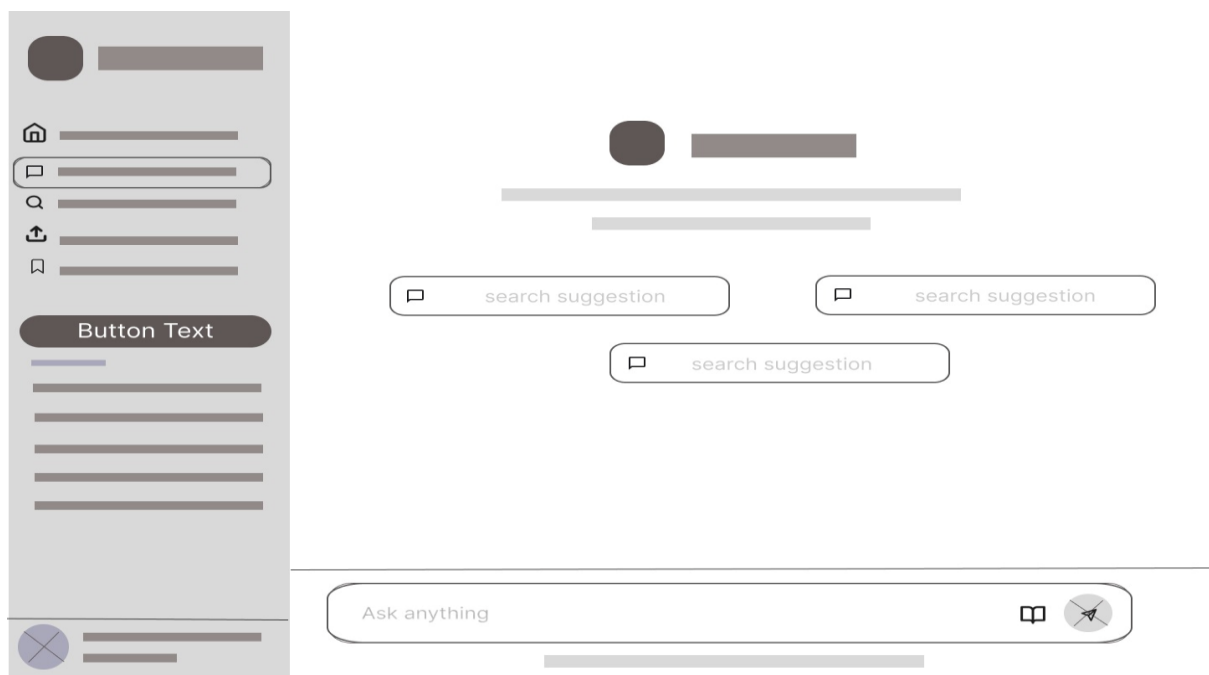


Figure 1.10: Low-fidelity wireframe for the main chat screen.

This wireframe represents the main chat interface of the application. It includes the navigation sidebar, suggested research questions, and a message input area where users can ask questions and receive AI-assisted responses.

Search Screen



Figure 1.11: Low-fidelity wireframe for the search screen.

This wireframe shows the search feature where users can search for academic topics, papers, or stored research content. It includes a search input, suggested results, filter options, and a results area.

Upload Screen



Figure 1.12: Low-fidelity wireframe for the upload screen.

This wireframe shows the file upload screen where users can upload research papers or PDF documents. The central upload area helps users easily add files to the system for later analysis and interaction.

Document Viewer and Chat Screen

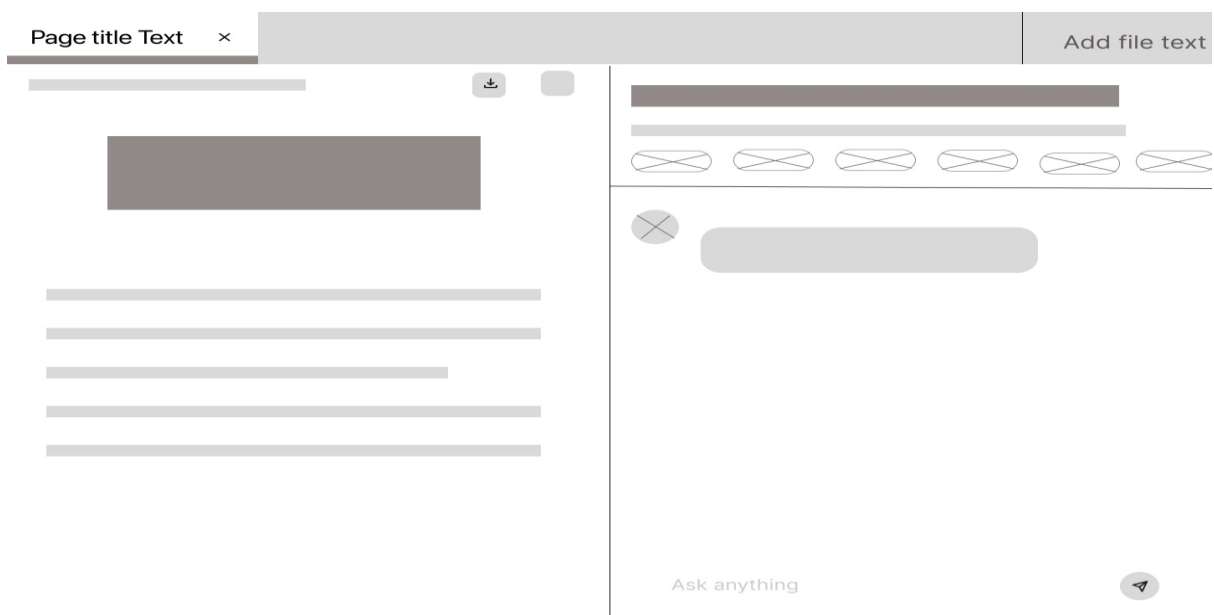


Figure 1.13: Low-fidelity wireframe for the document viewer and chat screen.

This wireframe shows the document viewer layout after a file is uploaded. The left side displays the selected document, while the right side provides a chat panel where users can ask questions about the uploaded file content.

paper Details / Summary Modal



Figure 1.14: Low-fidelity wireframe for the paper details and summary modal.

This wireframe represents a detailed paper view or summary modal. It provides an area for displaying paper information such as title, abstract, and related details, while the right side can display generated content, summaries, or analysis results.

Citation / Document Reading Screen



Figure 1.15: Low-fidelity wireframe for the citation and document reading screen.

This wireframe shows the citation and document reading screen. It includes a central document reading area, a navigation sidebar, and a right-side panel for related citation information, summaries, or supporting research content.

Section 02: Frontend

2.1 Frontend Overview

The frontend of **DATA2DASH** is designed as a web-based AI research dashboard that helps users search, understand, summarize, visualize, cite, and listen to academic research content in one place. The main goal of the interface is to reduce tool-switching between Google Scholar, ChatGPT, PDF readers, and document editors by providing a unified research platform. The project presentation describes DATA2DASH as an **AI-powered multi-agent research platform** that moves users from “searching and reading” toward “understanding and discovering.”

The frontend is implemented using **React 18**, **TypeScript**, and **Vite**, with **Tailwind CSS** for styling and responsive layout. The GitHub full-stack repository also lists React Router, Zustand, Framer Motion, Headless UI, Lucide React, and i18next as part of the frontend stack.

The interface includes main screens such as authentication, home/dashboard, academic search, upload/PDF interaction, citation generation, and AI agent features. The frontend connects the user with different AI agents such as the Intelligent Search Agent, Summarizer, Citation Agent, Knowledge Graph Agent, Multimodal RAG Agent, Critical Reviewer, Podcast/Audio Agent, and the updated **Video Agent**.

2.2 Frontend Technologies Used

Technology	Version / Type	Usage in DATA2DASH
React	React 18	Building the main web user interface
TypeScript	TypeScript 5	Adding type safety and improving code reliability
Vite	Vite build tool	Running and building the frontend project
Tailwind CSS	CSS framework	Styling pages, cards, buttons, layouts, and responsiveness
React Router DOM	Routing library	Navigation between pages such as Home, Search, Upload, and Citation
Zustand	State management	Managing frontend application state
Framer Motion	Animation library	Adding smooth UI animations and transitions
Headless UI	UI component library	Creating accessible interface components
Lucide React	Icon library	Displaying icons for search, upload, citation, audio, video, and agents
i18next / react-i18next	Internationalization	Supporting future multilingual interface options

2.3 Frontend Folder Structure

<https://github.com/Data2Dash/Data2Dash-frontend/tree/main/src/components>

2.4 Design Colors and Typography

Type	Color Name	Hex Code	Usage
Primary	Brand Black	#0F0F0E	Navigation bar, main buttons, logo, dark sections
Secondary	Forest Green	#1B4332	Active states, selected elements, important highlights
Accent	Accent Green	#40916C	Icons, highlights, progress bars, active UI elements
Background	Cream	#F5F3ED	Main page background
Secondary Background	Warm Gray	#EAE7DF	Secondary buttons, cards, dividers
Neutral	White	#FFFFFF	Cards, panels, document areas
Text	Muted Text	#6B6B65	Subtitles, captions, metadata, secondary labels

The typography used in the UX design is based mainly on **DM Sans**, which is used for headings, body text, buttons, labels, and captions. **DM Mono** is used for technical details such as metadata, color codes, and small system labels.

Font	Usage
DM Sans	Headings, paragraphs, labels, buttons, navigation
DM Mono	Technical labels, metadata, hex codes, small system text

2.5 Main Frontend Screens

The frontend contains multiple screens that cover the main user workflow. The UX design file includes wireframes for core screens such as **Login**, **Chat**, **Upload / PDF Viewer**, **Search**, and **Citation**

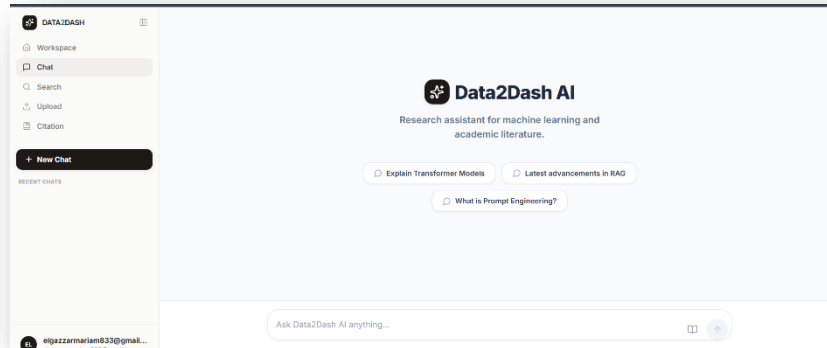
1. Authentication Screen

The authentication screen includes both Sign in and Sign up interfaces. It allows users to create an account, sign in, continue with Google, and use email/password authentication. **Sign in and Sign up screenshots.**

2. Sidebar Navigation

The application includes a fixed left sidebar that gives users access to the main sections of the platform: Workspace, Chat, Search, Upload, and Citation.

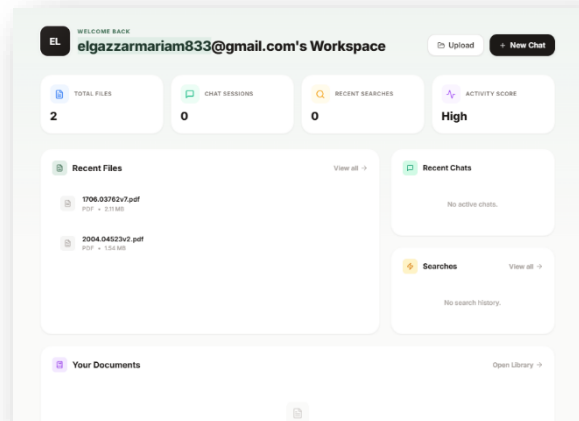
Workspace, Chat, Search, Upload, and Citation sidebar screenshots.



3. Workspace Dashboard

The workspace dashboard acts as the user's main home page. It shows total files, chat sessions, recent searches, activity score, recent files, recent chats, and search history.

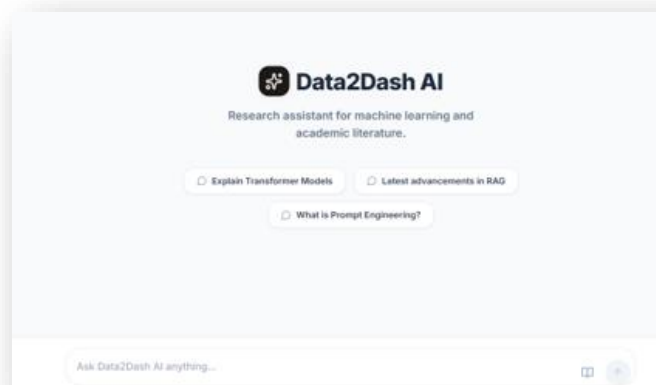
Workspace dashboard screenshot.



4. Chat Screen

The chat screen is titled Data2Dash AI and allows users to ask research questions. It also includes suggested prompts such as explaining transformer models, latest advancements in RAG, and prompt engineering.

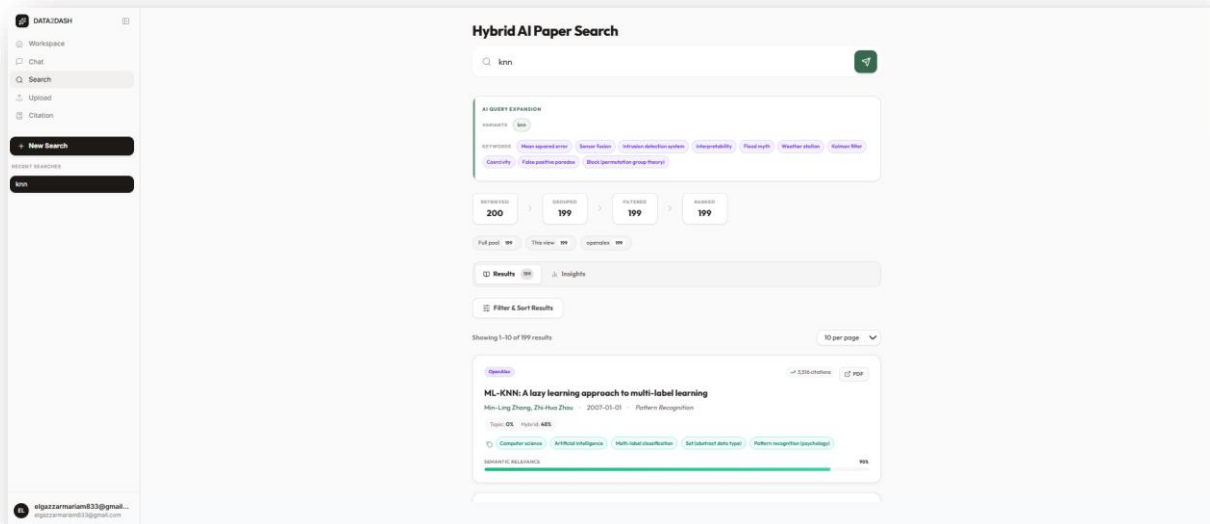
Data2Dash AI chat screenshot.



5. Hybrid AI Paper Search Screen

The search screen provides an academic search interface with a search input and suggested topics such as RAG, Transformers, GAN, BERT, and diffusion models. It supports AI-assisted paper search.

Hybrid AI Paper Search screenshot.

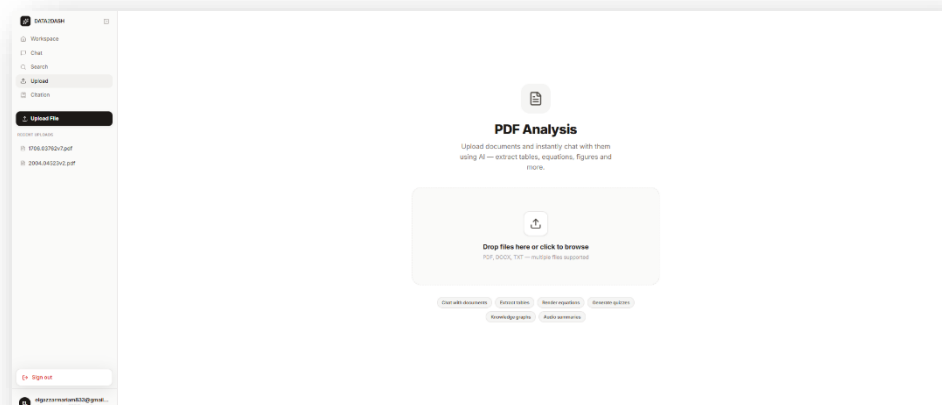


6. Upload Screen

The upload screen is titled PDF Analysis. It allows users to upload documents such as PDF, DOCX, and TXT files for AI-based analysis.

Evidence from screenshots: PDF Analysis upload screenshot.

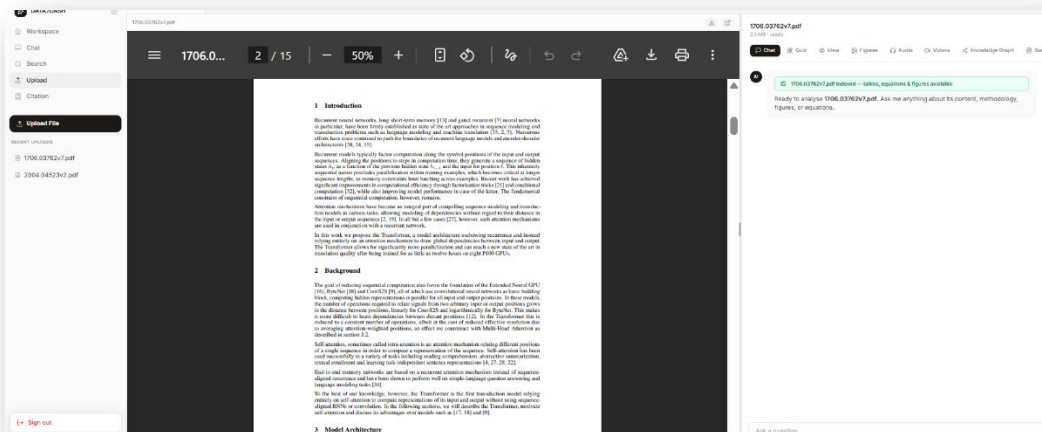
Add the upload page screenshot.



7. Document Analysis Workspace

After uploading a PDF, the document workspace shows the PDF viewer and multiple AI document tools. These include Chat, Quiz, View, Figures, Audio, Videos, Knowledge Graph, and Summarize.

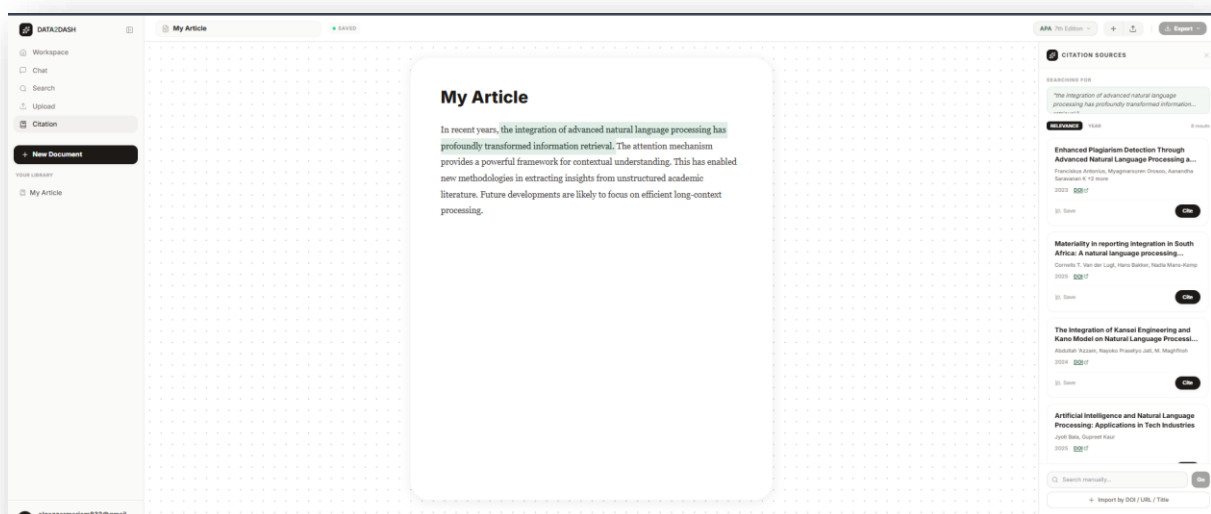
Uploaded PDF analysis screenshots.



8. Citation Editor

The citation screen provides an article editor where users can write academic text, select a citation format such as APA 7th Edition, export content, and view word count and character count.

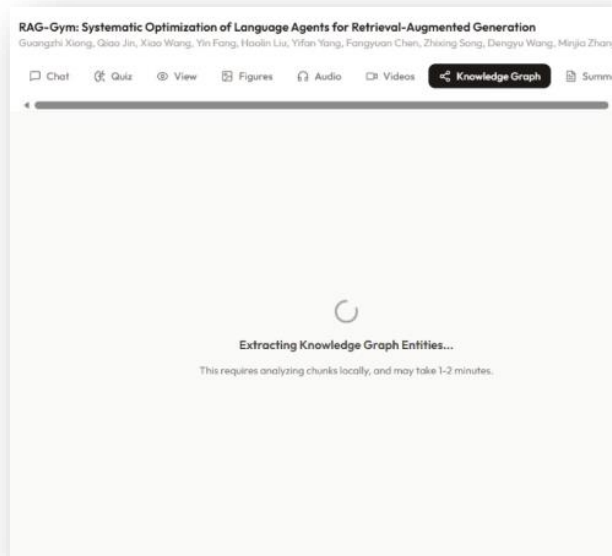
My Article citation editor screenshot.



9. Knowledge Graph Processing

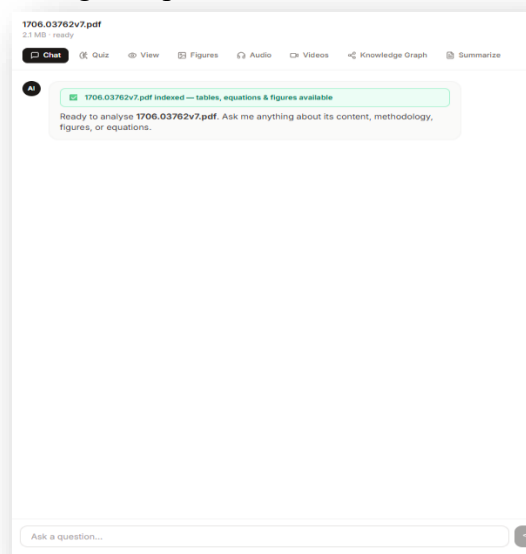
The document analysis workspace includes a Knowledge Graph feature. The screenshot shows the system extracting knowledge graph entities from an uploaded document. This supports the project's goal of helping users understand relationships inside research papers.

Knowledge graph extraction screenshot.



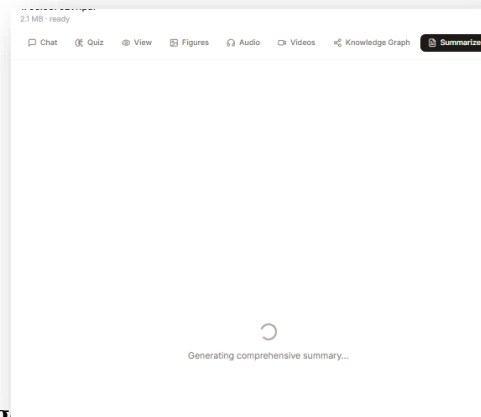
10. Multimodal RAG / PDF Analysis Agent

The upload and PDF analysis screen allows users to upload documents and interact with them using AI. The document workspace includes tabs for Chat, Quiz, View, Figures, Audio, Videos, Knowledge Graph, and Summarize.



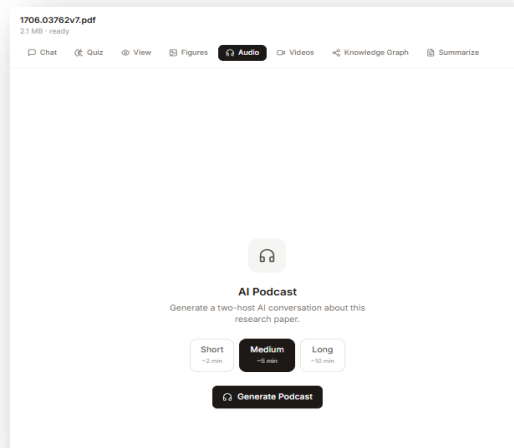
11. Paper Summarizer Agent

The summarizer feature helps users generate a short summary of an uploaded research paper. In the document workspace, the Summarize tab shows that summarization is part of the document analysis tools.



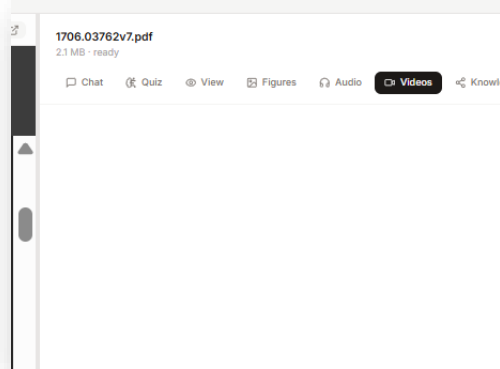
12. Audio / Podcast Agent

The audio feature allows users to listen to paper explanations or audio summaries. In the document workspace, the Audio tab shows that audio support is included as part of the platform.



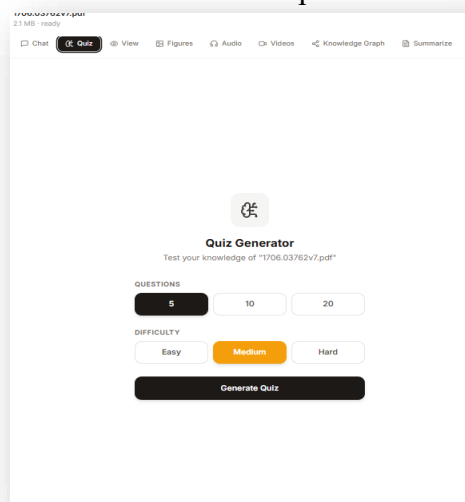
13. Video Agent

The Video Agent replaces the original YouTube Video Recommender idea. Instead of only recommending videos, it generates a simple presentation about a selected topic and explains it using audio. In the document workspace, the Videos tab shows that video support is included in the frontend.



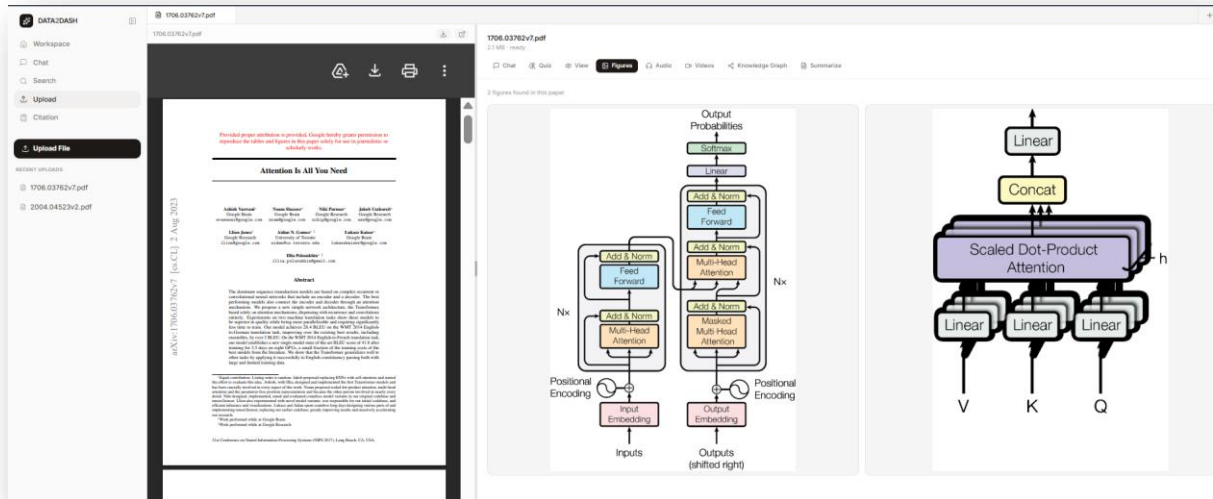
14. Quiz Agent

The document workspace includes a Quiz tab, which can be used to generate questions based on the uploaded document. This helps users test their understanding of the paper content.



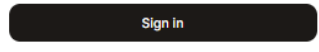
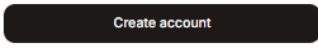
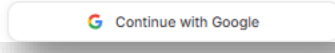
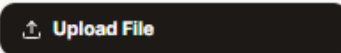
15. Figure Extractor Agent

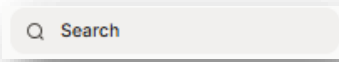
The Figure Extractor Agent is included in the document analysis workspace through the Figures tab. This feature is used to extract or display figures from uploaded research papers, helping users understand diagrams, charts, and visual content without manually searching through the full PDF. It supports the DATA2DASH goal of making academic papers easier to understand through text, visual, and audio-based tools.



2.6 Custom Buttons

DATA2DASH uses reusable button components to keep the interface consistent. Primary actions such as signing in, searching, generating, and exporting use strong visual emphasis, while secondary actions use lighter colors. Icons are used beside buttons where needed to make actions easier to understand.

Component Name	Style	Used In	Description	Screenshot to Add
SignInButton	Black filled button with rounded corners	Authentication screen	Used to sign in to an existing account	
CreateAccountButton	Wide black primary button	Sign-up form	Submits the create account form	
GoogleButton	White outlined button with Google icon	Authentication form	Allows users to continue with Google	
UploadButton	Black filled button with upload icon	Workspace and Upload pages	Used to upload files	
NewChatButton	Black filled button with plus icon	Chat page	Starts a new chat session	

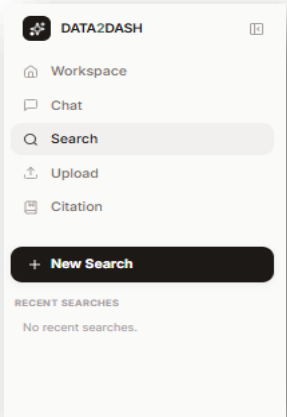
NewSearchButton	filled button with plus icon	Search page	Starts a new paper search	
ExportButton	Gray/toolbar button with export icon	Citation editor	Exports article or citation content	
SendButton	Icon button	Chat and search inputs	Sends chat messages or search queries	

2.7 Custom Components

The frontend is built around reusable components such as navigation bars, agent cards, search result cards, upload panels, chat inputs, citation editors, and video generation panels. This improves consistency and makes the project easier to extend with new agents in the future

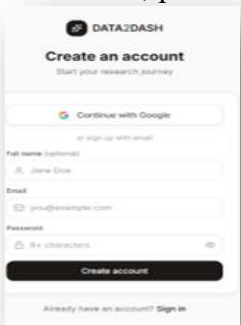
1. 1 Sidebar Component

Used across all main application screens. It displays the DATA2DASH logo, main navigation items, page action button, and user profile area.



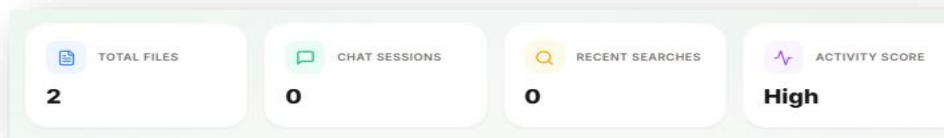
2. Authentication Form Component

It contains the Google login button, email field, password field, and account creation fields.



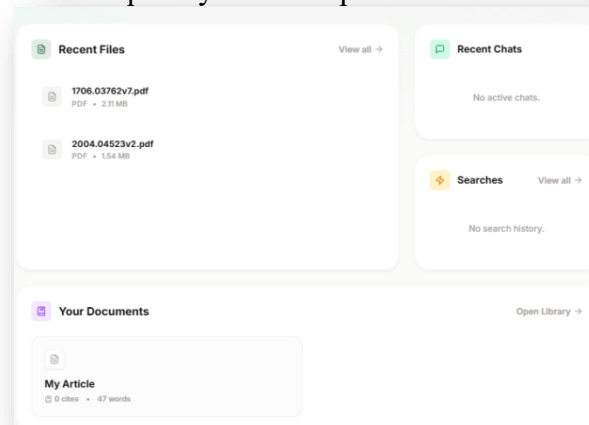
3. Workspace Dashboard Cards

Used in the Workspace dashboard. These cards display important user activity data such as total files, chat sessions, recent searches, and activity score.



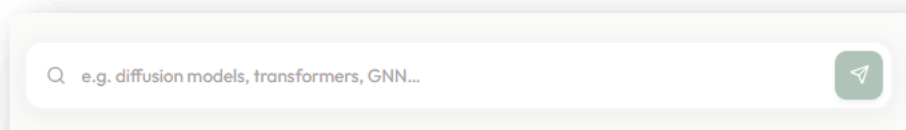
4. Workspace Activity Panels

Used in the Workspace dashboard. These panels show recent files, recent chats, and search history, so the user can quickly return to previous work.



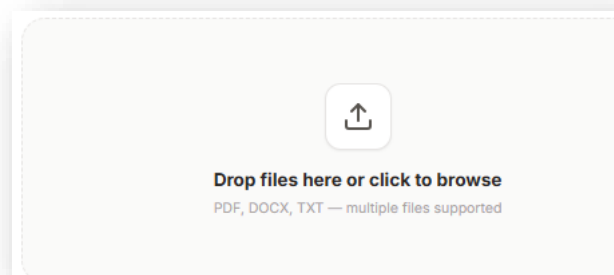
5. Search Input and Suggestion Chips

Used in the Chat and Search pages. The search input allows the user to type research questions or paper search queries, while the suggestion chips provide quick topics such as RAG, Transformers, GAN, and BERT.



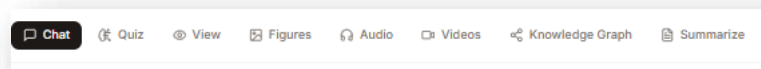
6. Upload Dropzone Component

Used in the Upload page. It allows users to drag and drop files or browse files from their device. It supports document upload for analysis.



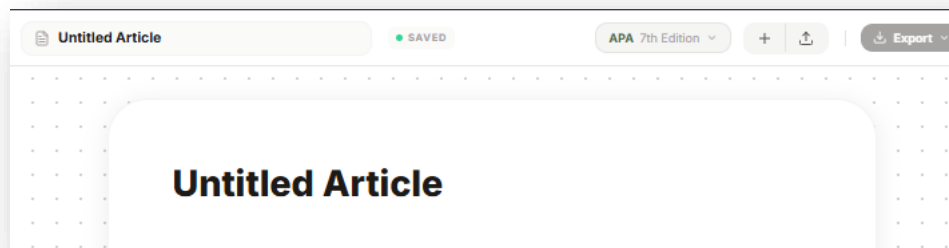
7. Document Tabs Component

Used in the PDF analysis workspace. It provides access to document tools such as Chat, Quiz, View, Figures, Audio, Videos, Knowledge Graph, and Summarize.



8. Citation Editor Component

Used in the Citation page. It provides an academic writing editor with citation format selection, export option, save status, and word/character count.



2.8 AI Agents Displayed in the Frontend

The DATA2DASH presentation lists nine intelligent agents, including Intelligent Search, Paper Summarizer, Knowledge Graph, Research Timeline, Multimodal RAG, AI Podcast Generator, Video Recommender, Citation Agent, and Critical Reviewer.

In the final implementation, the **YouTube Video Recommender** was updated to a more useful **Video Agent**. Instead of only recommending YouTube videos, the Video Agent generates a simple presentation about a topic and explains it using audio.

Agent	Frontend Page / Component	Description
Intelligent Search Agent	Search Page	Finds relevant academic papers using semantic search
Paper Summarizer	Summary Component	Generates concise summaries of research papers

Knowledge Graph Agent	Visualization Page	Shows relationships between papers, concepts, and citations
Research Timeline Agent	Timeline Component	Shows the development of a research topic over time
Multimodal RAG Agent	Upload / PDF Chat Page	Explains text, equations, graphs, and tables from uploaded documents
AI Podcast Generator	Audio / Listen Page	Converts paper content into audio explanations
Video Agent	Video Agent Page	Generates a simple presentation about a topic and explains it using audio
Citation Agent	Citation Page	Generates APA, IEEE, and other citation formats
Critical Reviewer	Review Page	Reviews methodology, limitations, and paper quality

2.10 Responsiveness

Since the frontend uses Tailwind CSS and a component-based React structure, the interface can be designed responsively across different screen sizes.

Device Size	Responsive Behavior to Test / Show in Report
Desktop	Show the full dashboard layout with navigation, cards, side panels, and multi-column sections
Tablet	Show that cards and sections resize into fewer columns
Mobile	Show that components stack vertically, forms become full-width, and navigation is simplified

2.11 Required Frontend Features

Required Feature	How DATA2DASH Includes It
Sign In / Sign Up	Authentication screen with Google and email login
Navigation	Navbar links between Chat, Search, Upload, Citation, and other pages
Notification	Notifications can inform users when summaries, citations, presentations, or audio are ready
Responsive Design	Layout adapts to desktop, tablet, and mobile views
Dynamic Content	Search results, summaries, citations, uploaded documents, generated presentations, and audio outputs are loaded dynamically

2.12 Final Frontend Summary

The frontend of DATA2DASH is implemented as a React and TypeScript web application using Vite and Tailwind CSS. The project follows a professional folder structure with separate folders for API logic, assets, reusable components, layouts, pages, styles, and TypeScript types. The interface uses a consistent design system based on dark brand colors, green accents, cream backgrounds, and DM Sans typography. Main screens include authentication, dashboard, chat, academic search, upload/PDF viewer, citation, notifications, and the updated Video Agent. The frontend connects users to multiple AI agents such as Intelligent Search, Paper Summarizer, Knowledge Graph, Multimodal RAG, Citation Agent, Critical Reviewer, AI Podcast Generator, and Video Agent. The Video Agent improves the original YouTube recommender idea by generating a topic-based presentation and explaining it with audio, making the platform more interactive and useful for learning.

Section 03: Backend

3.1 Backend Overview

The backend of Data2Dash AI is responsible for handling user authentication, file uploads, chat sessions, document processing, and database storage. The system uses a Python backend connected to a local SQLite database named `data2dash.db`. The backend receives requests from the frontend, processes them, stores the required data in the database, and sends dynamic responses back to the user interface.

The backend runs locally on:

`http://localhost:8000`

while the frontend runs on:

`http://localhost:5173`

This separation allows the frontend to focus on user interaction, while the backend manages logic, data storage, authentication, and AI-related operations.

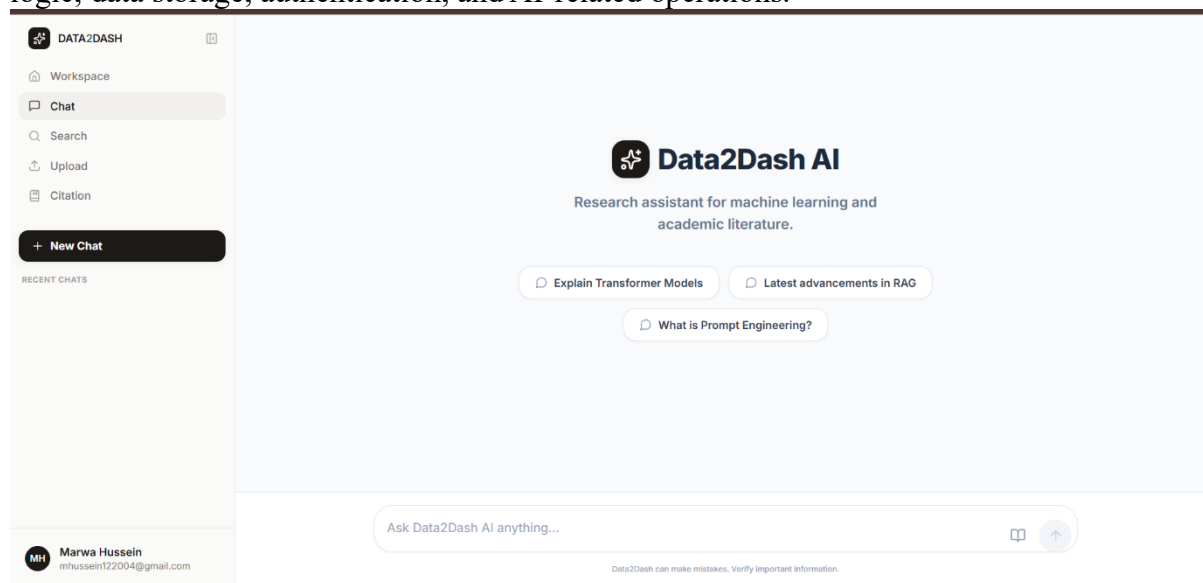


Figure 3.1: Data2Dash AI frontend connected to the backend service.

3.2 Backend Technologies

The following table shows the main backend technologies used in the project.

Technology	Usage in the Backend
Python	Main programming language used to build the backend logic.
FastAPI	Backend framework used to create API endpoints and handle frontend requests.
Uvicorn	ASGI server used to run the FastAPI backend locally.
SQLite	Local relational database used to store application records during development.
SQLAlchemy	Object Relational Mapper used to connect Python backend models with database tables.
DB Browser for SQLite	Tool used to inspect, test, and verify database records.
Python-dotenv	Used to load environment variables from the <code>.env</code> file.

JWT Authentication	Used to manage secure authenticated user sessions.
Password Hashing	Used to store user passwords securely as hashed values instead of plain text.
File Processing	Used to handle uploaded PDF files and store file metadata.

3.3 Database Structure

The application uses an SQLite database file named: data2dash.db

The database file is located inside the backend folder:

backend_python/data2dash.db

The database contains multiple tables that support the main features of the application. Each table is responsible for storing a specific type of data.

The main tables are:

- users**
- workspaces**
- files**
- file_versions**
- chat_sessions**
- chat_messages**
- search_history**
- documents**

The purpose of each table is described below:

Table Name	Purpose
users	Stores user account information such as email, name, hashed password, and account status.
workspaces	Stores workspace information related to each user.
files	Stores uploaded file metadata such as filename, file type, size, and storage path.
file_versions	Stores versioning information for uploaded files if file versions are created.
chat_sessions	Stores chat session information such as title, user ID, workspace ID, and timestamps.
chat_messages	Stores the actual user and AI messages inside each chat session.
search_history	Stores user search queries and related metadata.
documents	Stores generated or processed document content, citations, style, and word count.

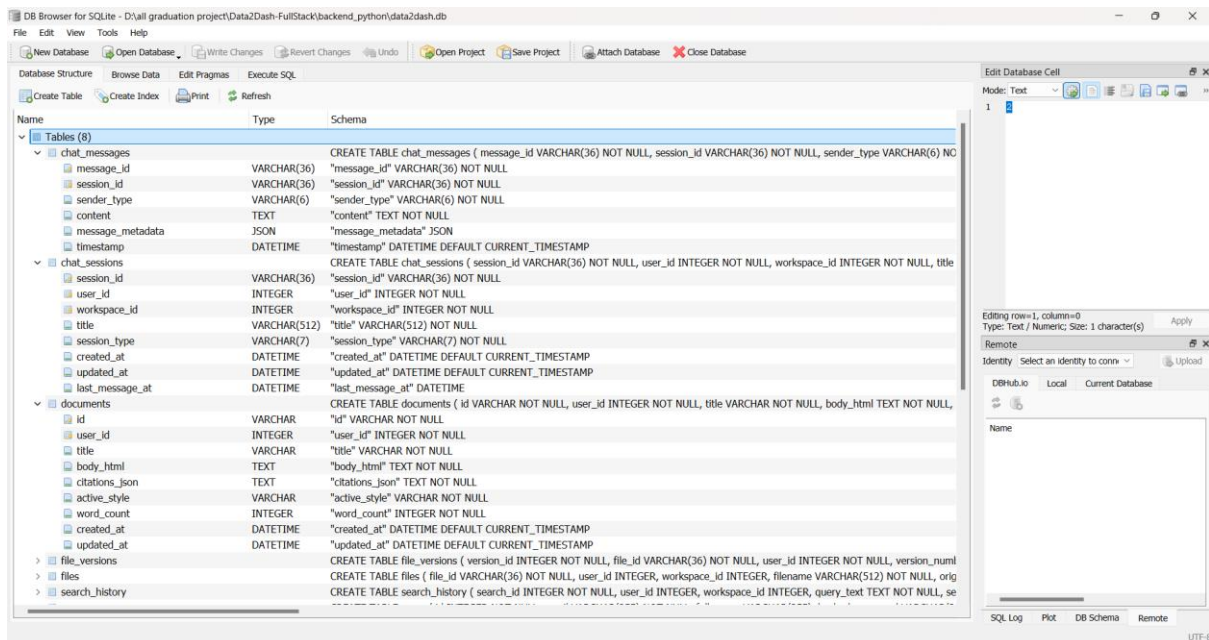


Figure 3.2: Database schema showing the main SQLite tables used by the backend.

3.4 User Authentication and User Records

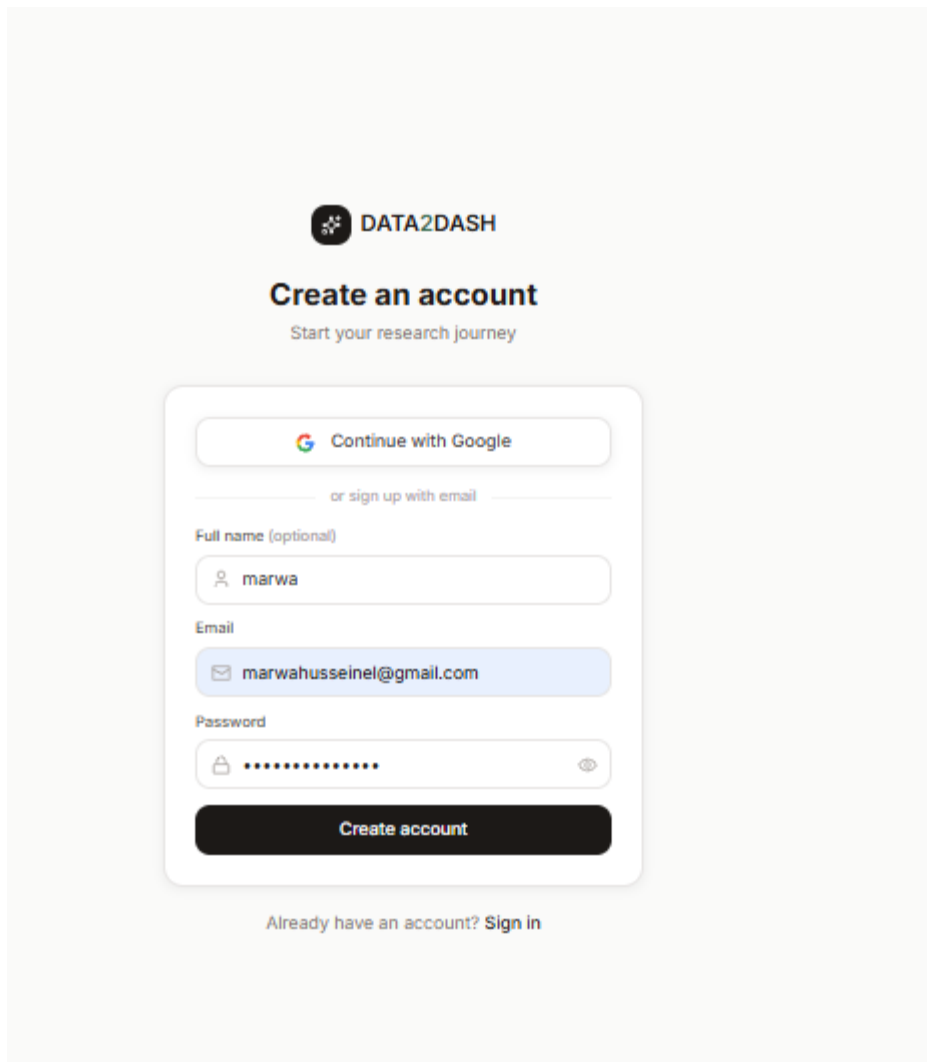
The application includes a sign-up and sign-in system. When a new user creates an account from the frontend, the backend receives the submitted data and stores a new record in the users table.

The users table contains fields such as:

- id**
- email**
- full_name**
- hashed_password**
- google_id**
- avatar_url**
- is_active**
- created_at**
- updated_at**
- last_login_at**

The password is not stored as plain text. Instead, the backend stores it as a hashed value in the hashed_password column. This improves security because the original password is not directly visible in the database.

During testing, two user accounts were created successfully. After creating the accounts from the frontend, the records appeared in the users table inside the SQLite database. This proves that the registration form is connected to the backend and database correctly.



DATA2DASH

Create an account

Start your research journey

[Continue with Google](#)

or sign up with email

Full name (optional)

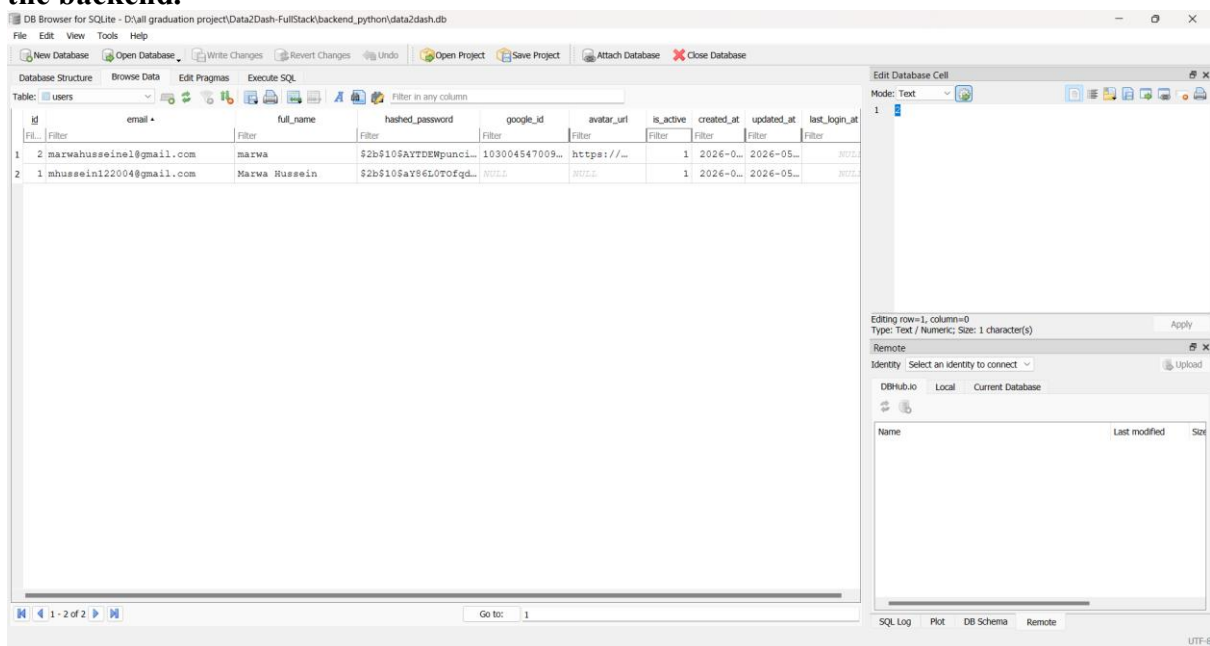
Email

Password

[Create account](#)

Already have an account? [Sign in](#)

Figure 3.3: User registration form filled with test account data before submitting it to the backend.



DB Browser for SQLite - D:\ai graduation project\Data2Dash-FullStack\backend_python\data2dash.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

Table: users

id	email	full_name	hashed_password	google_id	avatar_url	is_active	created_at	updated_at	last_login_at
1	marwahussein1@gmail.com	marwa	\$2b\$10\$AYTDEMjunc1...	103004547009...	https://...	1	2026-0...	2026-05...	NOT L...
2	mhussein12204@gmail.com	Marwa Hussein	\$2b\$10\$aY86L0rofqd...	NOT L...	NOT L...	1	2026-0...	2026-05...	NOT L...

Go to: 1

Editing row=1, column=0
Type: Text / Numeric; Size: 1 character(s) Apply

Remote Identity Select an identity to connect Upload

DBHub.io Local Current Database

Name Last modified Size

SQL Log Plot DB Schema Remote UTF-8

Caption:
Figure 3.4: User records stored in the SQLite database after account creation.

Explanation:

The records in the users table confirm that the backend successfully receives account data from the frontend and saves it in the database. The hashed_password field shows that passwords are stored securely as hashed values instead of plain text.

3.5 Workspaces

Each user has a workspace that organizes their activity inside the application. The workspaces table links users with their related workspace data.

The workspaces table includes fields such as:

Column Name	Description
id	Unique identifier for each workspace record.
user_id	References the user who owns this workspace.
name	Stores the workspace name.
created_at	Stores the date and time when the workspace was created.

During testing, the SQL count query showed that two workspace records were created. This means that when users are created or logged in, workspace-related data is also stored in the database.

This helps the backend organize user-specific files, chats, and documents under the correct user workspace.

3.6 Chat Sessions and Chat Messages

The chat feature is one of the main dynamic parts of the Data2Dash AI application. When the user sends a question from the frontend, the backend receives the request, creates or updates a chat session, stores the conversation in the database, and returns the AI response to be displayed in the chat interface.

The chat system mainly depends on two database tables: chat_sessions and chat_messages.

Chat Sessions Table

The chat_sessions table stores general information about each conversation. Each chat session belongs to a specific user and workspace.

Table 3.4: Structure of the chat_sessions table

Column Name	Description
session_id	Unique identifier for each chat session.
user_id	References the user who created the chat session.
workspace_id	References the workspace related to the chat session.
title	Stores the generated or selected title of the conversation.
session_type	Defines the type of chat session.
created_at	Stores the date and time when the chat session was created.
updated_at	Stores the date and time when the chat session was last updated.

last_message_at	Stores the date and time of the last message in the session.
------------------------	---

Chat Messages Table

The chat_messages table stores the actual messages inside each chat session. It includes both the user messages and the AI responses. This allows the application to retrieve and display previous conversations dynamically.

Table 3.5: Structure of the chat_messages table

Column Name	Description
message_id	Unique identifier for each message.
session_id	References the chat session that the message belongs to.
sender_type	Identifies whether the message was sent by the user or by the AI assistant.
content	Stores the text content of the message.
message_metadata	Stores additional message-related information if available.
timestamp	Stores the date and time when the message was created.

For example, during testing, the user asked:

Explain Transformer Models

After submitting this question, the backend created a new chat session and stored the related messages in the database. The chat session also appeared dynamically in the frontend sidebar under Recent Chats.

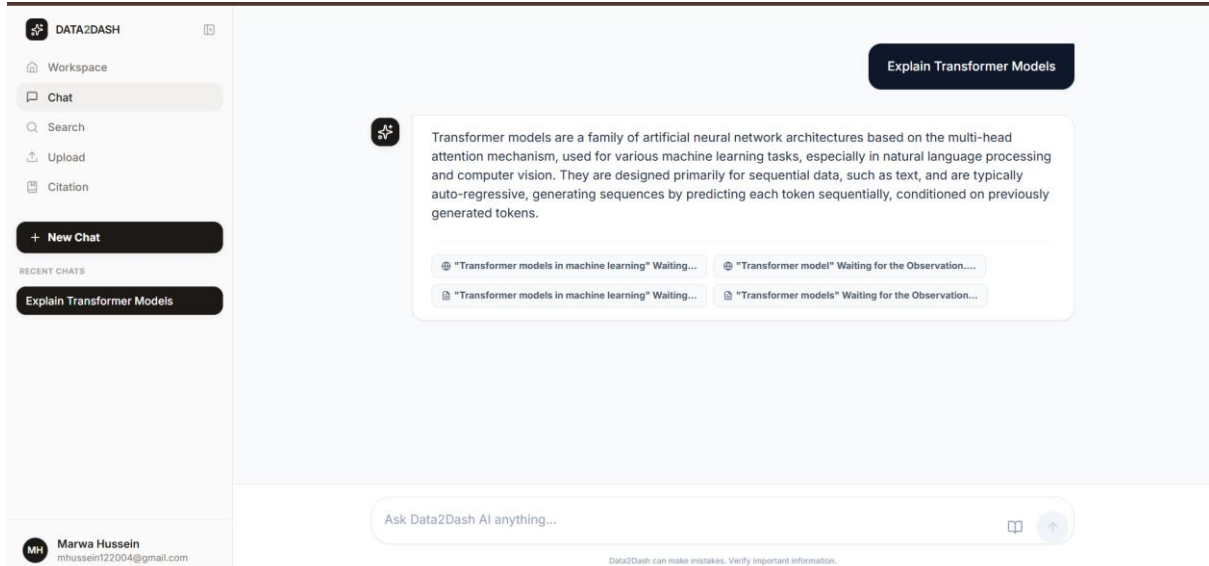


Figure 3.5: Dynamic AI chat response rendered in the frontend.

session_id	user_id	workspace_id	title	session_type	created_at	updated_at	last_message_at
cb680963-27a0-4406-...	1	1	Explain ...	ai	2026-05-07 14:14:57	2026-05-07...	2026-05-07 ...

Figure 3.6: Chat sessions saved in the database after user interaction.

The user also tested a follow-up question, such as:

explain in more details multi-head attention mechanism

This follow-up interaction proves that the chat feature supports continuous conversation and that the frontend updates dynamically based on user input and backend responses.

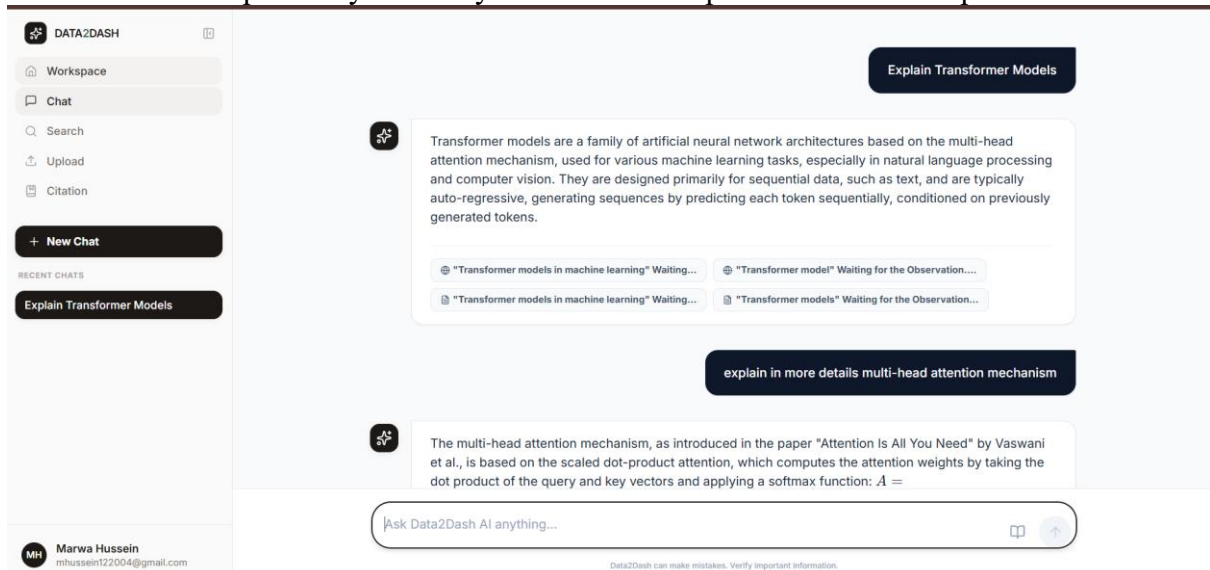


Figure 3.7: Follow-up chat interaction rendered dynamically in the application.

Explanation

The chat screenshots and database records prove that the chat content is dynamic. When the user sends a message, the frontend displays the response in the chat interface, while the backend stores the session and messages in the database. This makes it possible for the application to show previous conversations again in the sidebar and maintain chat history for each user.

This confirms that the chat feature is not static; it is connected to backend database records through the `chat_sessions` and `chat_messages` tables.

3.7 File Upload and File Records

The backend handles file uploads by receiving files from the frontend, saving their metadata, and storing the related information in the SQLite database. When a user uploads a PDF file, the backend inserts a new record into the files table.

Table 3.6: Structure of the files Table

Column Name	Description
file_id	Unique identifier for each uploaded file.
user_id	References the user who uploaded the file.
workspace_id	References the workspace where the file belongs.
filename	Stores the system-generated file name.
original_name	Stores the original name of the uploaded file.
file_type	Stores the type of the uploaded file, such as PDF.
mime_type	Stores the MIME type of the file, such as application/pdf.
size_bytes	Stores the file size in bytes.
storage_path	Stores the path where the uploaded file is saved.

During testing, a PDF file named: 1810.04805v2.pdf was uploaded successfully. The file appeared in the frontend upload page, and the backend inserted a new record into the files table.

Table 3.7: File Upload Test Result

Test Item	Result
Tested Feature	PDF file upload
Uploaded File Name	1810.04805v2.pdf
Frontend Result	The uploaded PDF appeared inside the Data2Dash AI interface.
Backend Result	File metadata was inserted into the files table.
Database Evidence	A new record appeared in DB Browser inside the files table.

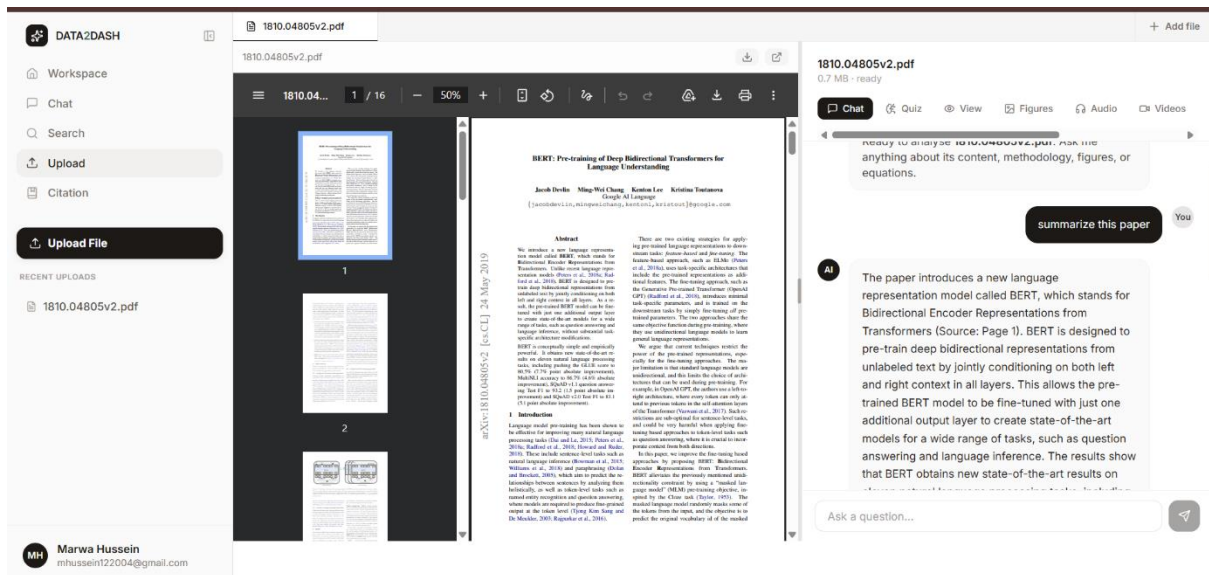


Figure 3.8: Uploaded PDF file displayed inside the Data2Dash AI interface.

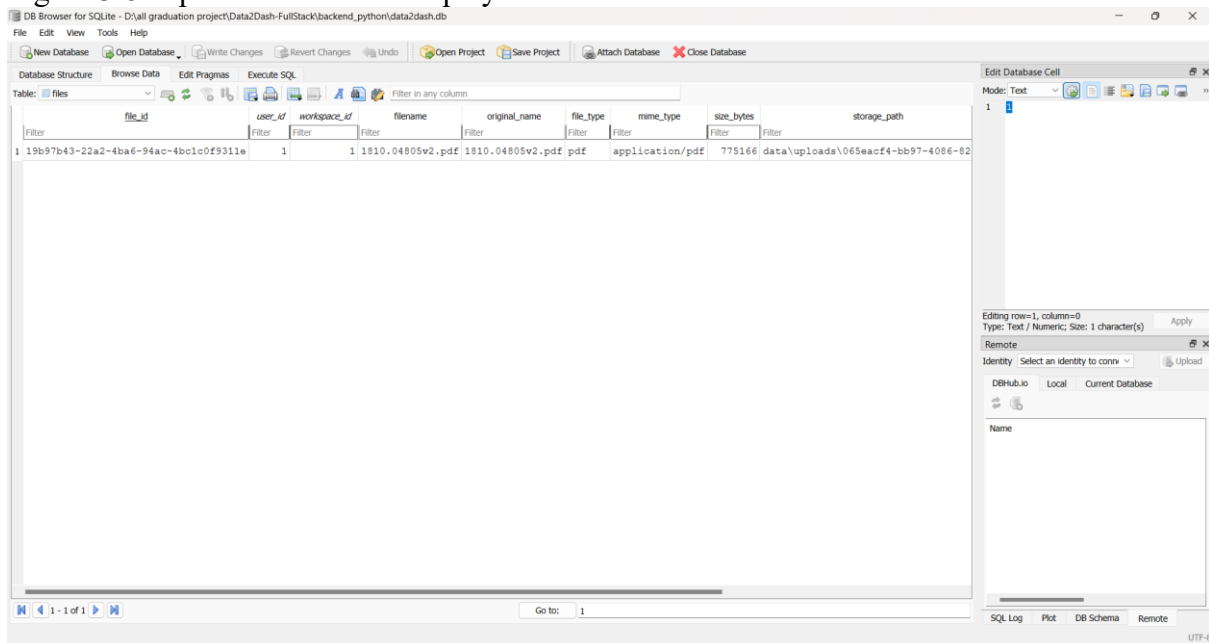


Figure 3.9: Uploaded file metadata stored in the files table.

The file upload test confirms that the backend stores uploaded file metadata correctly. The frontend displays the uploaded PDF, while the database stores the file information for future retrieval.

3.8 Document Records

The documents table stores document-related data created or processed by the application. This table can store generated document content, citation information, formatting style, word count, and timestamps.

Table 3.8: Structure of the documents Table

Column Name	Description
id	Unique identifier for each document record.
user_id	References the user who owns or created the document.

title	Stores the document title.
body_html	Stores the document body content in HTML format.
citations_json	Stores citation data in JSON format.
active_style	Stores the selected formatting or citation style.
word_count	Stores the number of words in the document.
created_at	Stores the date and time when the document was created.
updated_at	Stores the date and time when the document was last updated.

During testing, the documents table contained two records with the title: **My Article**

Table 3.9: Document Records Test Result

Test Item	Result
Tested Table	documents
Number of Records	2
Example Document Title	My Article
Backend Result	The backend stored document-related data in the database.
Database Evidence	Two document records appeared in DB Browser.

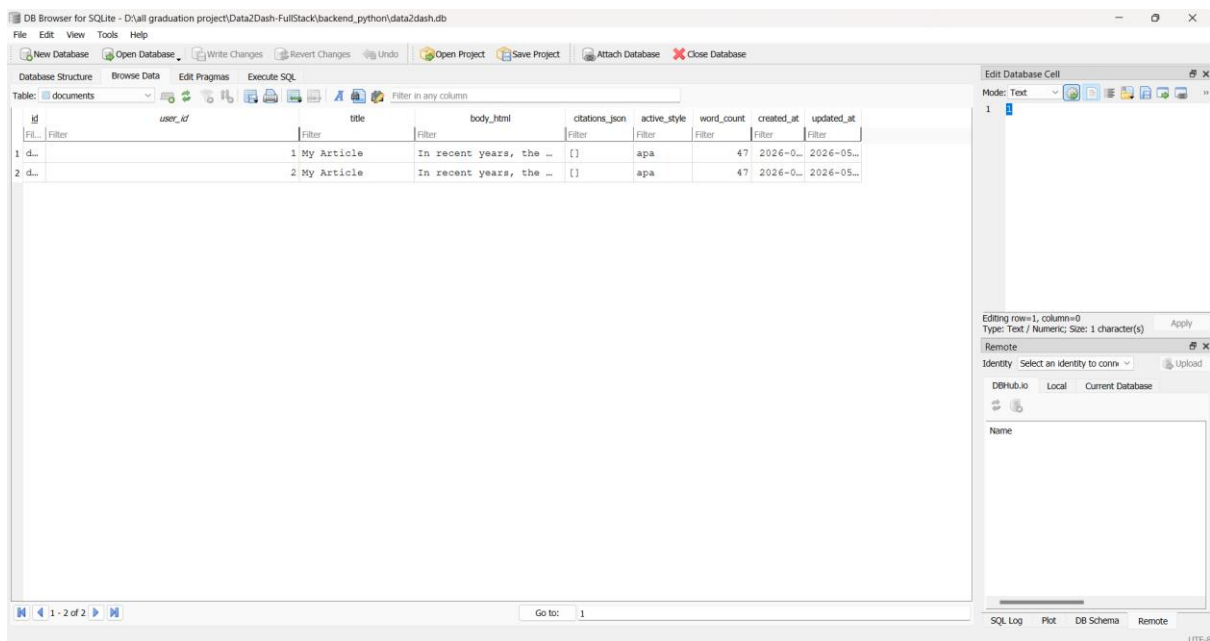


Figure 3.10: Document records stored in the SQLite database.

The document records show that the backend supports storing structured document content. This content can later be retrieved and displayed dynamically in the frontend

3.9 Database Testing Using SQL Query

To verify backend database integration, an SQL query was executed using DB Browser for SQLite. The purpose of this query was to count the number of records in each main database table after testing the application features.

Table 3.10: SQL Query Purpose

Item	Description
Tool Used	DB Browser for SQLite
Tested Database	data2dash.db
Query Purpose	Count the number of records in each main database table.
Reason	To prove that frontend actions were stored in the backend database.

The query used was:

```
SELECT 'users' AS table_name, COUNT(*) AS rows_count FROM users
UNION ALL
SELECT 'workspaces', COUNT(*) FROM workspaces
UNION ALL
SELECT 'files', COUNT(*) FROM files
UNION ALL
SELECT 'file_versions', COUNT(*) FROM file_versions
UNION ALL
SELECT 'chat_sessions', COUNT(*) FROM chat_sessions
UNION ALL
SELECT 'chat_messages', COUNT(*) FROM chat_messages
UNION ALL
SELECT 'search_history', COUNT(*) FROM search_history
UNION ALL
SELECT 'documents', COUNT(*) FROM documents;
```

Table 3.11: Database Record Count Result

Table Name	Number of Records	Explanation
users	2	Two user accounts were created during testing.
workspaces	2	Two workspace records were created for the users.
files	1	One PDF file was uploaded and stored.
file_versions	0	No file versioning operation was performed during this test.
chat_sessions	3	Three chat sessions were created.
chat_messages	8	Eight chat messages were stored, including user messages and AI responses.
search_history	0	The search feature was not tested or not recorded during this specific test.

documents	2	Two document records were stored in the database.
------------------	----------	--

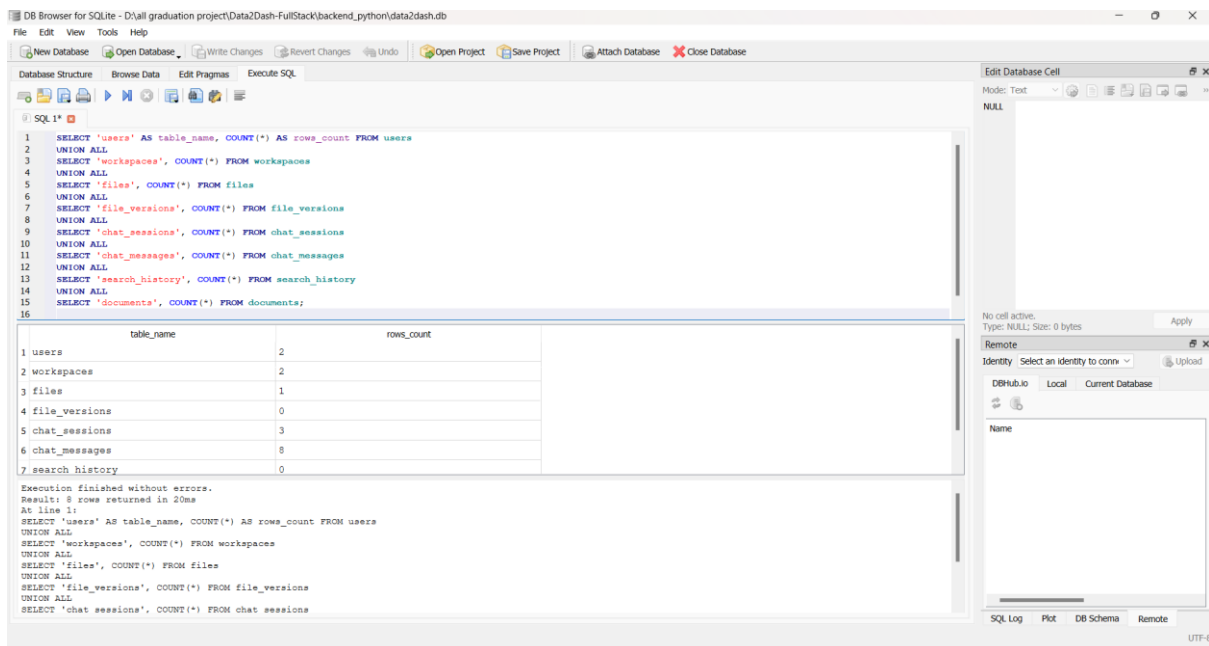


Figure 3.11: SQL query result showing record counts for each backend database table. The SQL count result provides clear evidence that the backend is storing data in multiple database tables. It confirms that user registration, workspace creation, chat sessions, chat messages, file uploads, and document storage were successfully tested.

3.10 Dynamic Content Rendered from the Database

Several parts of the frontend are dynamic and depend on backend database records. This means the application does not only display static content. Instead, the interface changes according to the data stored and retrieved from the database.

Table 3.12: Examples of Dynamic Content Rendered from the Database

Dynamic Content	Related Database Table	Explanation
Logged-in user name and email	users	The sidebar displays the logged-in user's name and email based on stored user data.
Recent chat titles	chat_sessions	The sidebar displays recent chat titles retrieved from saved chat sessions.
Chat messages and AI responses	chat_messages	The chat interface displays stored user messages and AI responses.
Uploaded file list	files	Uploaded files are displayed based on file metadata stored in the database.

Document records	documents	Stored documents can be retrieved and displayed dynamically.
Workspace-related data	workspaces	User activity is organized under workspace records.

For example, after the user logs in, the sidebar displays the user name and email. These values are related to the stored user record. Also, when the user creates a new chat, the chat title appears under Recent Chats. This information is stored in the chat_sessions table and rendered dynamically in the frontend.

When the user uploads a file, the uploaded file appears in the upload section. The backend stores its metadata in the files table, and the frontend displays it to the user.

Table 3.13: Dynamic Content Evidence

Frontend Area	What Appears in the UI	Backend Evidence
Sidebar	User name and email	Stored in the users table.
Recent Chats	Chat titles such as “Explain Transformer Models”	Stored in the chat_sessions table.
Chat Interface	User questions and AI responses	Stored in the chat_messages table.
Upload Page	Uploaded PDF file	Stored in the files table.
Documents Section	Document title and content	Stored in the documents table.

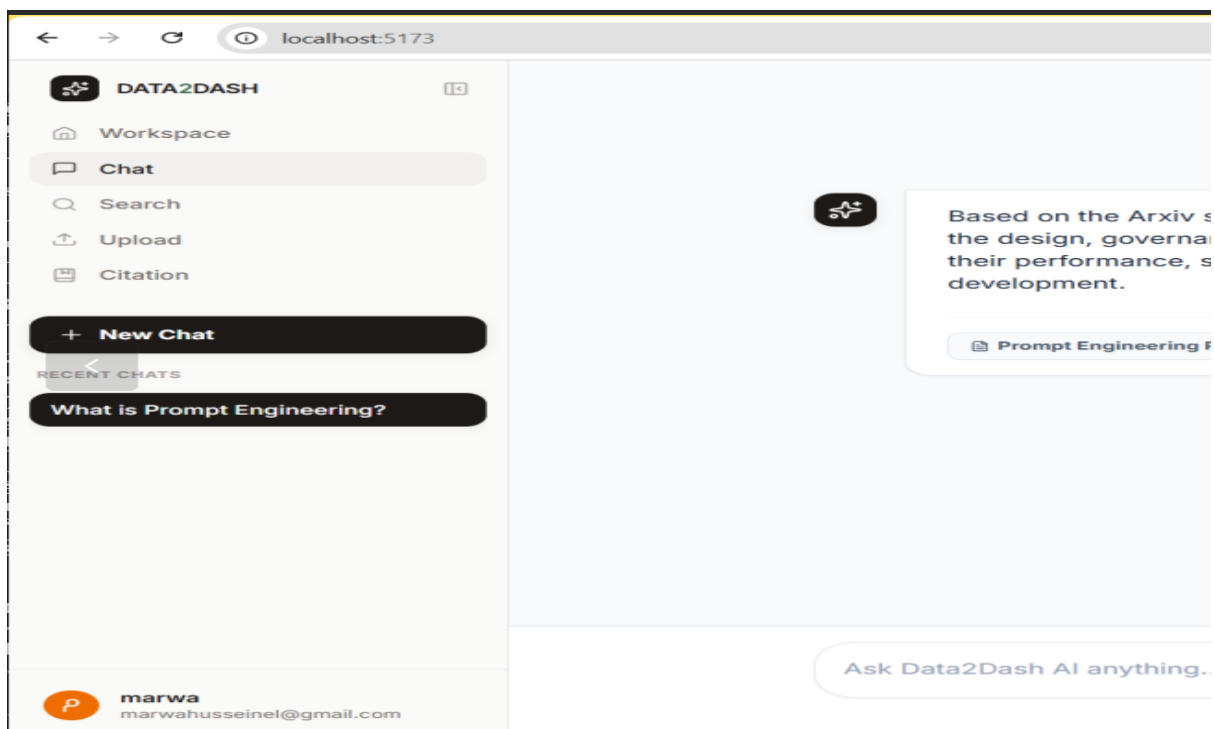


Figure 3.12: Dynamic user profile and recent chat content rendered from backend data.

This figure proves that the frontend is not only displaying static content. The sidebar changes based on the logged-in user and the user’s saved chat sessions. This confirms that dynamic content is being retrieved from backend data and displayed in the user interface.

Required Element	Should Be Visible?
User name at the bottom of the sidebar	Yes
User email	Yes
Recent Chats list	Yes
A chat title such as “Explain Transformer Models” or “What is Prompt Engineering?”	Yes

3.11 Evidence of Backend and Database Integration

The testing results prove that the frontend, backend, and database are connected successfully.

Table 3.14: Backend and Database Integration Evidence

Test Action	Frontend Result	Database Result
Create account	User account was created and displayed in the sidebar.	A new record was inserted into the users table.
Start chat	Chat response was displayed in the chat interface.	A new record was inserted into the chat_sessions table.
Send messages	User and AI messages were displayed in chat.	Messages were stored in the chat_messages table.
Upload PDF	Uploaded PDF appeared in the upload page.	File metadata was stored in the files table.
Store document content	Document data was available in the system.	Records were stored in the documents table.
Run SQL count query	Database records were summarized.	Row counts were displayed for each table.

This confirms that the application does not rely on static data only. The content changes based on user actions, and these actions are reflected in the backend database.

3.12 Backend Testing Summary

The backend was tested by performing real actions from the frontend and then checking the database records using DB Browser for SQLite.

Table 3.15: Backend Testing Steps

Step Number	Testing Step	Verification Method
1	Created user accounts from the frontend.	Checked the users table in DB Browser.
2	Verified that users were inserted into the database.	Confirmed user records appeared in the users table.
3	Sent chat questions from the frontend.	Checked the chat interface and database records.
4	Verified that chat sessions were inserted.	Checked the chat_sessions table.
5	Verified that chat messages were inserted.	Checked the chat_messages table.
6	Uploaded a PDF file.	Checked the upload page in the frontend.

7	Verified that file metadata was inserted.	Checked the files table.
8	Checked generated document records.	Checked the documents table.
9	Ran an SQL query to count records.	Used DB Browser Execute SQL tab.

Table 3.16: Backend Testing Summary Result

Tested Area	Status	Evidence
User authentication	Passed	User records appeared in the users table.
Workspace creation	Passed	Workspace records appeared in the workspaces table.
Chat sessions	Passed	Chat sessions appeared in the chat_sessions table.
Chat messages	Passed	Messages appeared in the chat_messages table.
File upload	Passed	Uploaded PDF metadata appeared in the files table.
Document records	Passed	Document records appeared in the documents table.
SQL database verification	Passed	Record count query returned results for all main tables.

The results confirm that the backend is working correctly. It receives data from the frontend, stores it in the SQLite database, retrieves dynamic content, and supports the main features of the application.

Conclusion

In conclusion, Data2Dash AI is a functional web-based research assistant that helps users upload academic papers, interact with documents, ask research questions, and organize their research work. The project followed a complete development process including UX research, problem definition, wireframing, frontend design, backend implementation, and database testing.

The frontend provides a responsive and organized interface with clear navigation between the main features, while the backend manages authentication, file storage, chat sessions, messages, documents, and user data. The SQLite database testing proved that the application stores real user actions and displays dynamic content from the backend.

Overall, the project successfully meets its main goal by connecting the frontend, backend, and database into one working system. The application can be further improved in the future by adding more advanced citation tools, better search history, stronger document analysis, and online deployment.